

Nama : Hafidz Alfiansyah

NPM : 50422636

Kelas : 4IA06

Judul & Deskripsi Aplikasi

Judul Proyek

Aplikasi Pemrograman Jaringan: Layanan Chatting, File Sharing, dan Server Video Streaming Menggunakan Jembatan Web Klien (Flask & Socket.IO) ke TCP Socket Server

Deskripsi Aplikasi

Aplikasi ini dirancang sebagai sistem komunikasi *client-server* terdistribusi yang memadukan keunggulan performa protokol raw TCP socket di sisi backend dengan kemudahan akses Web Browser di sisi frontend.

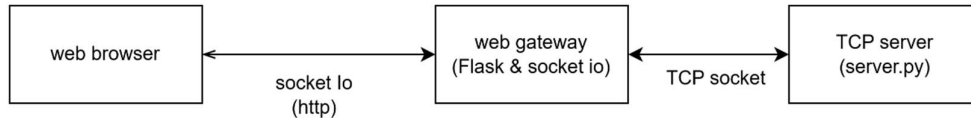
Aplikasi ini memiliki tiga fitur utama:

1. Layanan Chatting Real-Time: Media komunikasi teks antar-klien dengan mekanisme relay multi-client (broadcast).
2. Transfer File dengan File Explorer Server: Fitur untuk mengirim file dari browser klien ke server, di mana file disimpan secara permanen di direktori server (server_storage/). Sistem secara otomatis mendeteksi file yang diunggah dan menyediakannya dalam daftar file server yang dapat diunduh langsung lewat browser menggunakan protokol HTTP.
3. Server Video Streaming: Fitur pemutaran video secara streaming dari server ke browser klien. Klien dapat memilih file video yang tersedia di folder server_videos/ melalui daftar dropdown dinamis yang diperbarui otomatis, lalu memutarinya secara langsung di tag gambar browser.

Keamanan login dijamin menggunakan autentikasi OTP (One-Time Password) 6-digit yang dikirim langsung ke alamat email tujuan menggunakan server SMTP

Arsitektur Sistem

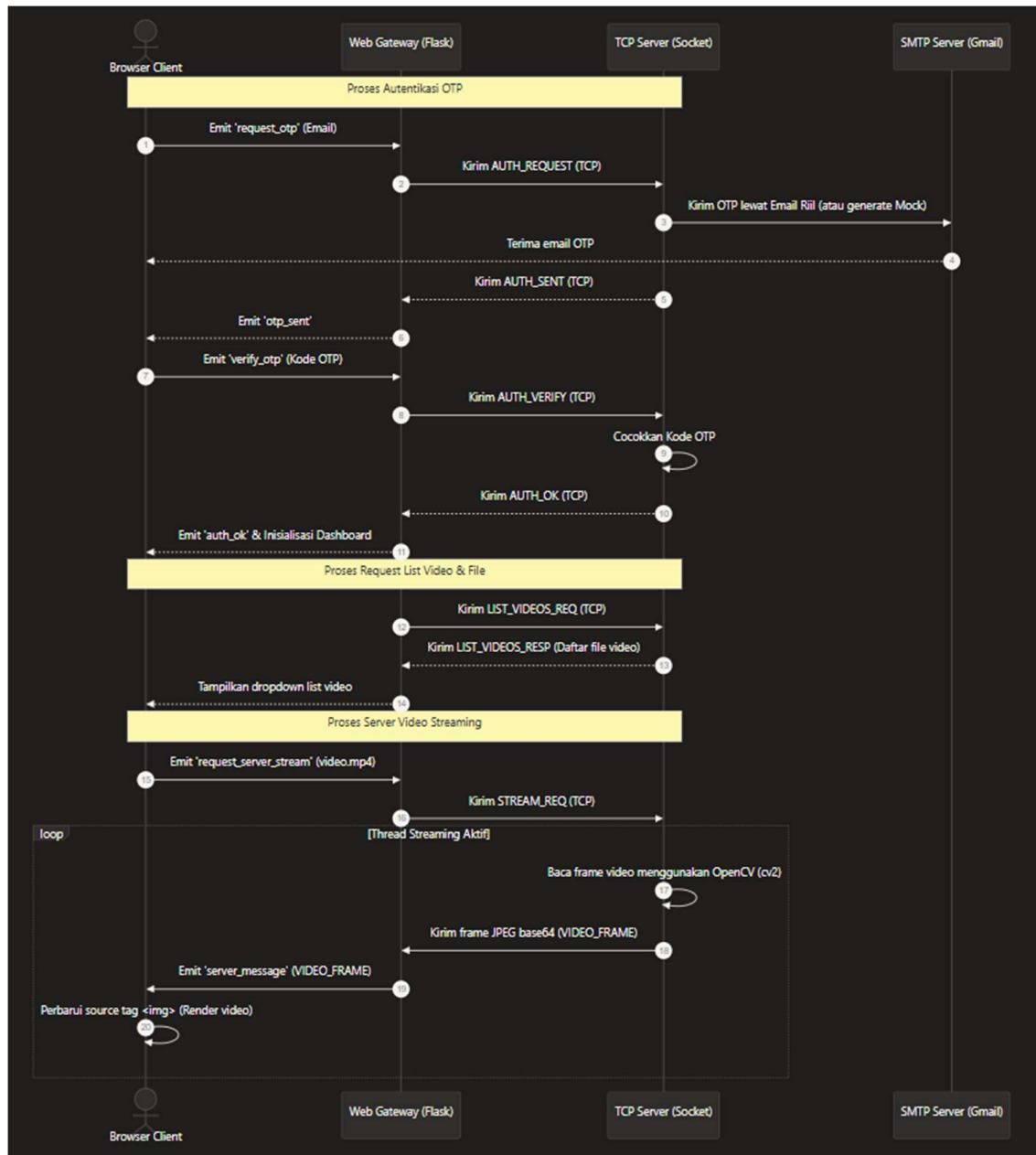
Aplikasi ini menggunakan arsitektur 3-Tier Bridge (Jembatan Tiga Lapis) untuk memfasilitasi komunikasi web browser ke raw TCP socket. Karena web browser tidak memiliki kapabilitas native untuk melakukan koneksi mentah ke TCP socket (server.py), dibuatlah Web Gateway (web_client.py) sebagai perantara:



1. **Frontend (Browser Client):** menggunakan HTML5, Vanilla CSS, dan Socket.IO client library. Tugasnya adalah menangkap input pengguna, membaca file chunk, memutar frame video, dan merender gelembung pesan.
2. **Gateway (Flask & Socket.IO Server):** Berfungsi sebagai penerjemah (translator) dua arah. Setiap koneksi browser dialokasikan satu TCP socket koneksi ke server utama. Gateway juga berfungsi sebagai HTTP server untuk mengunduh berkas fisik secara asinkron dari folder `server_storage/`.
3. **Backend (TCP Server):** Engine utama multithreaded yang bertugas mengelola perutean paket data chat, memverifikasi token OTP, menyimpan file fisik ke disk, dan memproses file video biner (.mp4) untuk dikirim menjadi frame-frame JPEG biner per koneksi secara streaming.

Flowchart & Diagram Komunikasi

Berikut adalah *Sequence Diagram* yang menggambarkan alur komunikasi dari autentikasi hingga proses request streaming video server:



Penjelasan Implementasi

A. Protokol Komunikasi Custom (protocol.py)

TCP socket bersifat *stream-based*, yang berarti data yang dikirim berturut-turut dapat bergabung atau terpotong (masalah *TCP sticky/fragmentation*). Untuk mengatasi hal ini, modul `protocol.py` membungkus setiap pesan JSON dengan header panjang data 4-byte (*length-prefixed framing*):

```
def send_msg(sock, msg_dict):
    data = json.dumps(msg_dict).encode('utf-8')
    header = len(data).to_bytes(4, byteorder='big')
    sock.sendall(header + data)

def recv_msg(sock):
    header = recv_all(sock, 4)
    if not header: return None
    length = int.from_bytes(header, byteorder='big')
    data = recv_all(sock, length)
    return json.loads(data.decode('utf-8'))
```

B. Otentikasi Berbasis OTP (auth_email.py)

Otentikasi menggunakan modul smtplib untuk terhubung ke SMTP server Gmail secara aman (starttls) :

```
server = smtplib.SMTP("smtp.gmail.com", 587)
server.starttls()
server.login(SMTP_USER, SMTP_PASS)
server.sendmail(SMTP_USER, email, text)
```

C. Pembagian Transfer File Asinkron (File Chunking)

Pada templates/index.html, file yang dipilih oleh pengguna dipecah menjadi bagian-bagian kecil (chunk) berukuran 60 KB menggunakan FileReader API di browser untuk menghindari kelebihan beban memori browser, kemudian dikirimkan secara beruntun:

```
function readNextChunk() {
    const slice = selectedFile.slice(offset, offset + CHUNK_SIZE);
    reader.onload = function(e) {
        const base64Data = arrayBufferToBase64(e.target.result);
        socket.emit('send_file_chunk', { filename: selectedFile.name, data_b64: base64Data });
        offset += CHUNK_SIZE;
        if (offset < selectedFile.size) { readNextChunk(); }
        else { socket.emit('send_file_end', { filename: selectedFile.name }); }
    };
    reader.readAsArrayBuffer(slice);
}
```

File biner ini diterima oleh server.py dan didecode, kemudian ditulis ke disk dalam direktori server_storage/

D. Server-Side Video Streaming (server.py)

Server memutar video dari disk secara frame-by-frame menggunakan OpenCV, memampatkannya menjadi format gambar JPEG biner, mengubahnya menjadi string base64, lalu mengirimkannya lewat TCP socket ke client:

```
cap = cv2.VideoCapture(video_path)
fps = cap.get(cv2.CAP_PROP_FPS) or 25
while not stop_event.is_set():
    ret, frame = cap.read()
    if not ret: break # Video selesai
    ok, buf = cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, 60])
    if ok:
        data_b64 = base64.b64encode(buf).decode('utf-8')
        protocol.send_msg(conn, {"type": "VIDEO_FRAME", "data_b64": data_b64})
        time.sleep(1.0 / fps)
cap.release()
```

Hasil Pengujian

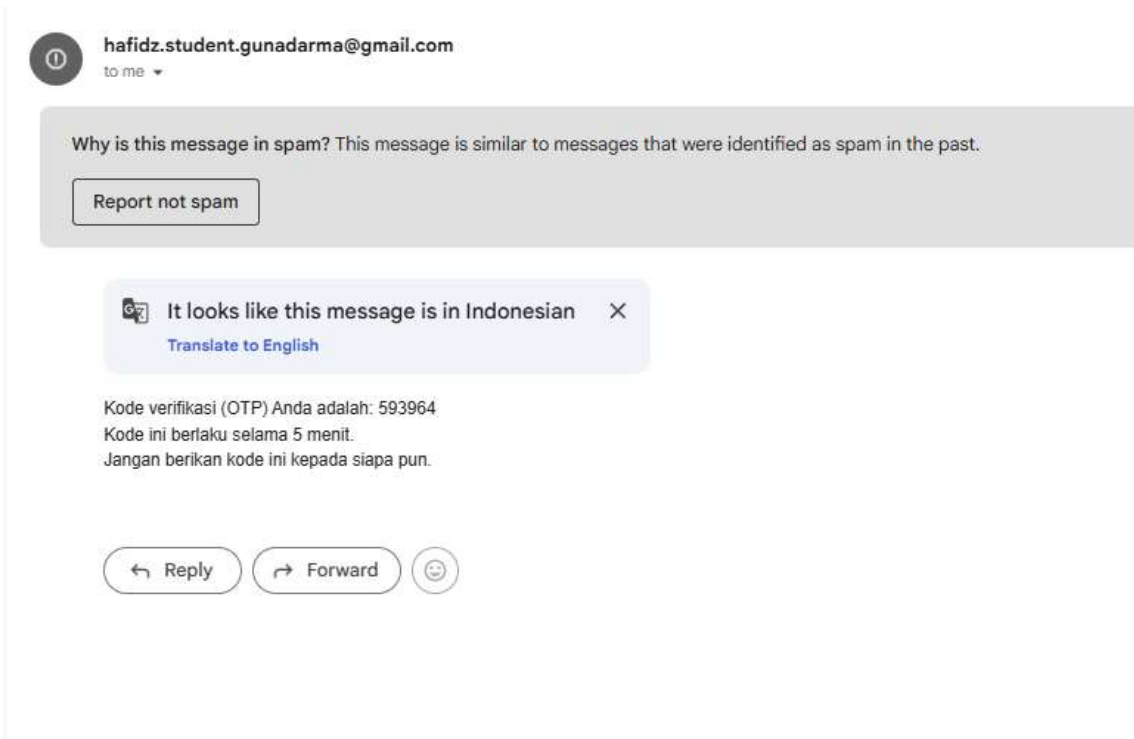
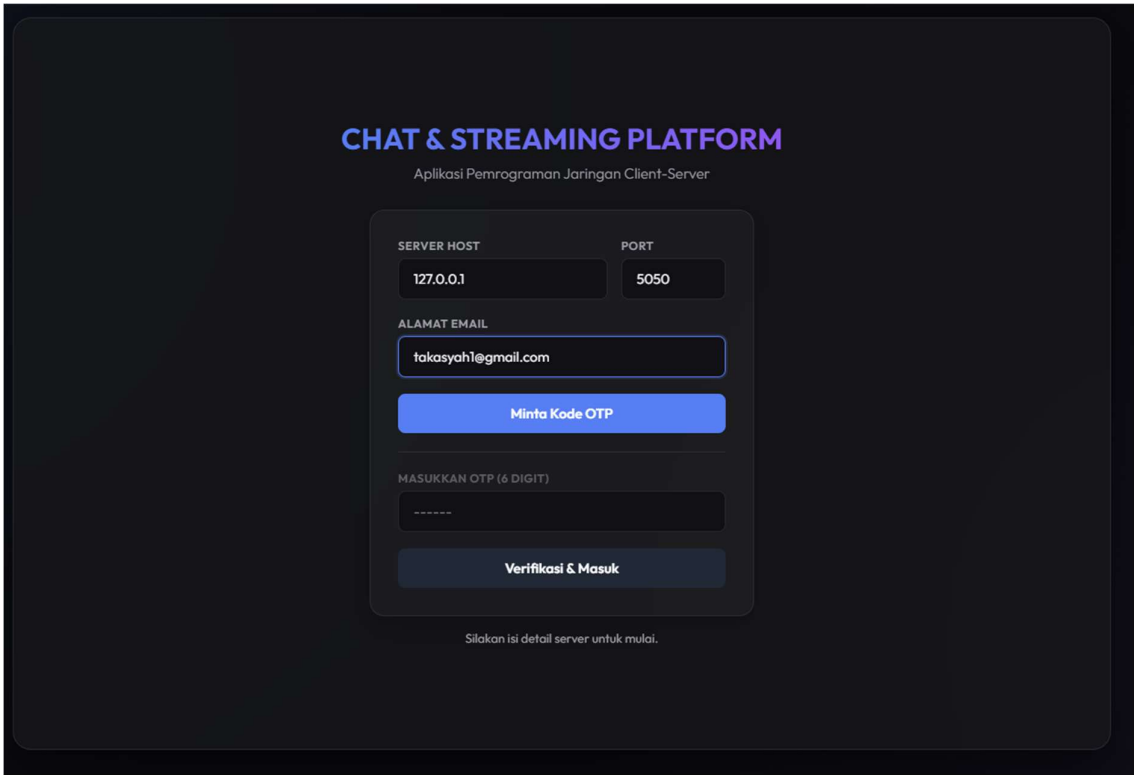
- Autentikasi OTP: OTP email terkirim dengan sukses menggunakan SMTP Google.
- Obrolan Chat: Pesan chat terkirim secara instant tanpa delay antar tab browser.
- Unggah File : File diunggah ke server dan muncul di tab sidebar File di Server.
- Daftar Video Server & Stream: Dropdown berhasil menampilkan file video di server secara otomatis. Saat tombol Request Stream ditekan, video terputar dengan lancar pada resolusi 320x240 di browser.

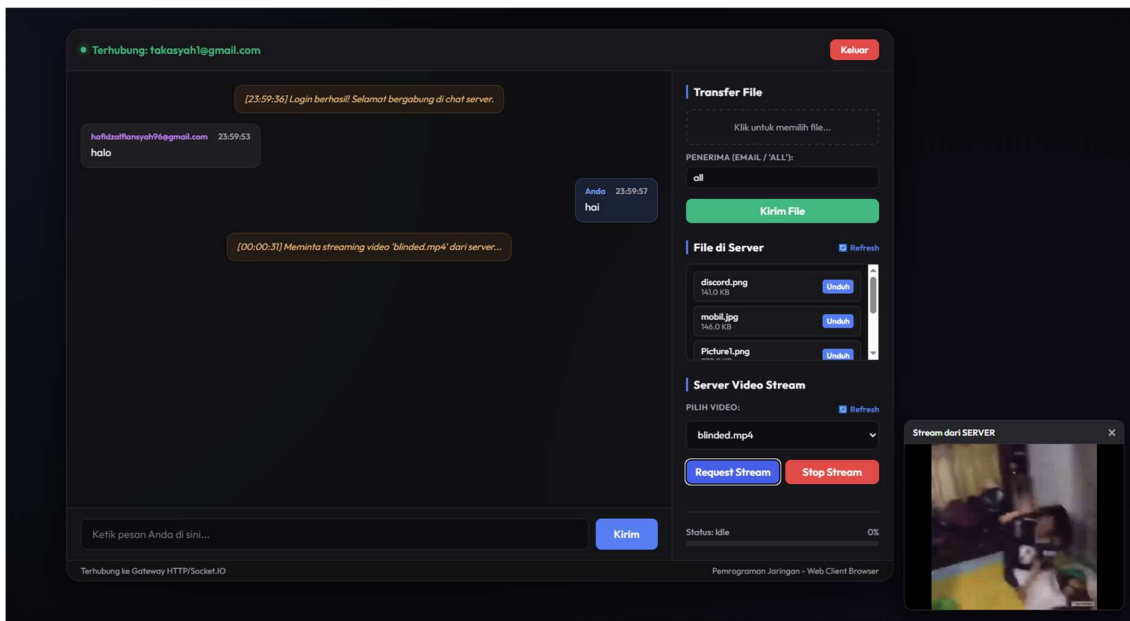
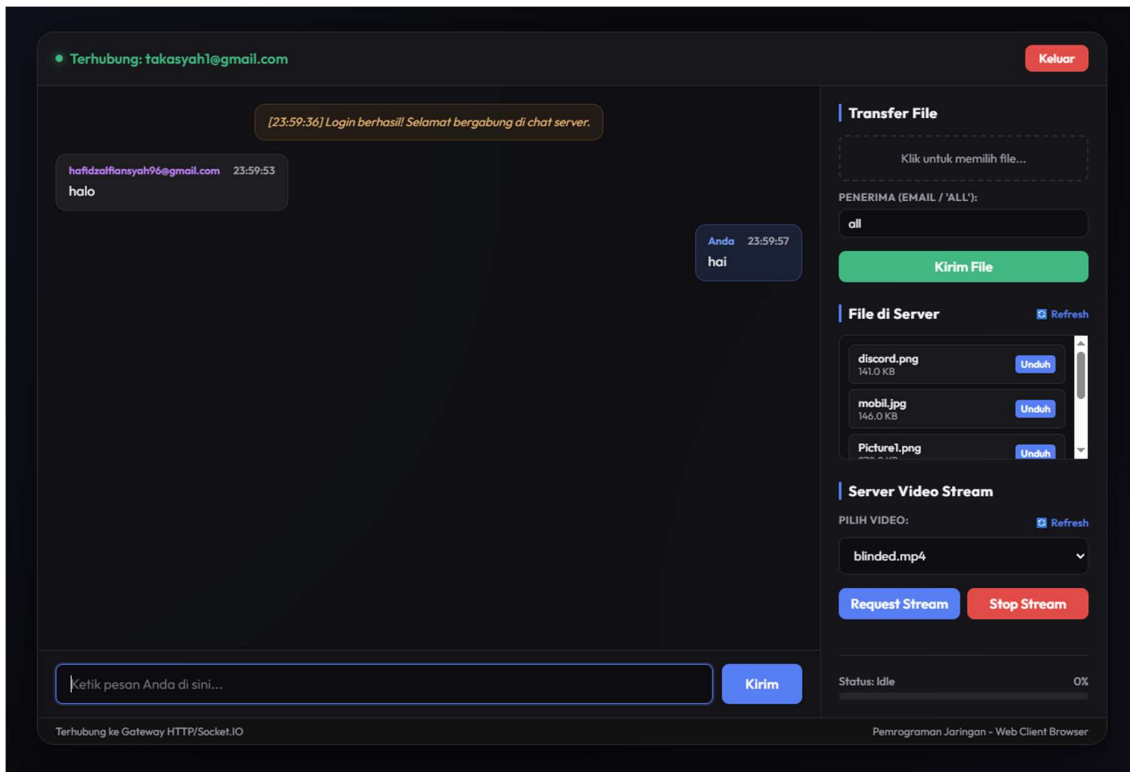
Kesimpulan

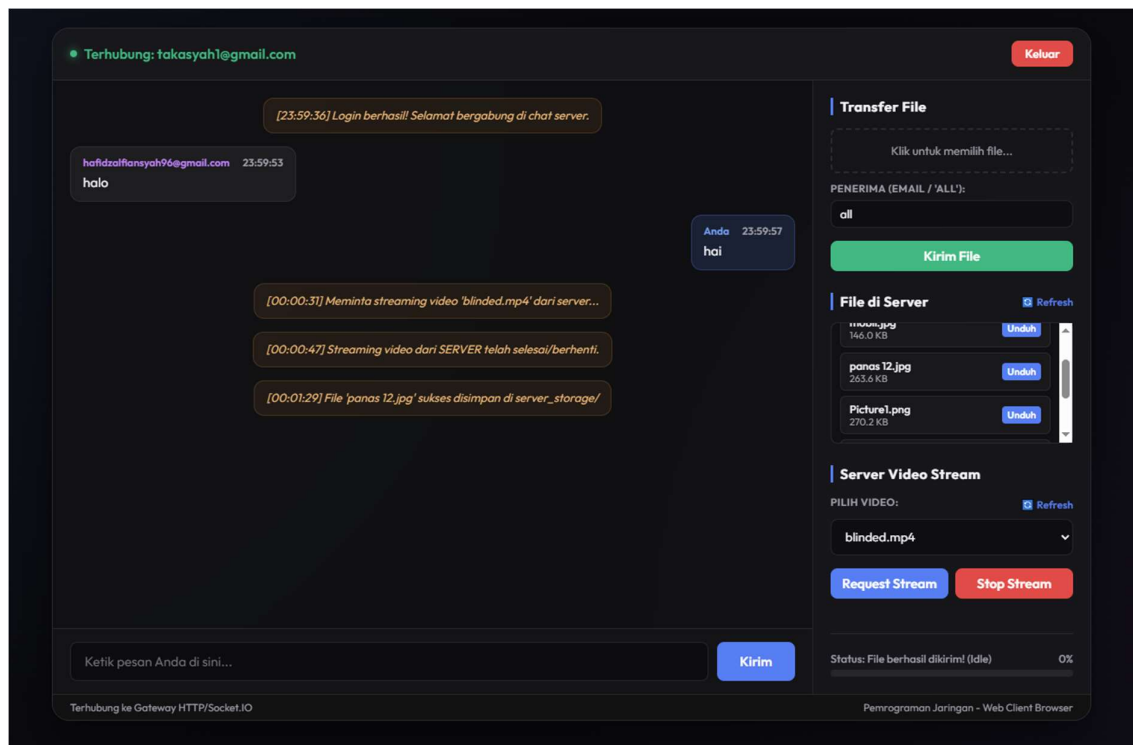
Proyek akhir Pemrograman Jaringan ini berhasil diimplementasikan dengan kesimpulan sebagai berikut:

1. Penggunaan Web Gateway berupa Flask-Socket.IO merupakan solusi terbaik untuk mengatasi keterbatasan native web browser yang tidak dapat mengakses raw TCP socket secara langsung.
2. Protokol custom length-prefixed framing terbukti sangat andal dalam menjamin keutuhan data biner (file transfer dan frame video) dan mencegah terjadinya pemotongan data akibat latensi jaringan.
3. Pembagian file menjadi chunk 60 KB di sisi browser terbukti efisien dalam menjaga pemakaian RAM browser dan kestabilan transmisi data biner berukuran besar.
4. Pemanfaatan HTTP route /download/<filename> di sisi Flask mempermudah klien mengunduh berkas langsung dari disk server tanpa membebani thread koneksi socket utama.

Screenshot







Jawaban Pertanyaan

1. Protokol jaringan apa yang digunakan pada aplikasi Anda? Jelaskan alasannya.
2. Bagaimana mekanisme komunikasi antara client dan server pada aplikasi yang dibuat?
3. Bagaimana aplikasi menangani kesalahan koneksi atau kegagalan pengiriman data?
4. Apa kelebihan dan kekurangan implementasi aplikasi jaringan yang Anda buat?
5. Jelaskan pengembangan yang dapat dilakukan agar aplikasi menjadi lebih aman, cepat, dan andal.

Jawab :

1. TCP (Transmission Control Protocol) : TCP menjamin pengiriman paket data secara andal (*reliable*), berurutan (*in-order delivery*), dan bebas dari error (melalui mekanisme checksum, acknowledgment, dan retransmisi). Hal ini sangat krusial untuk transfer file fisik dan pertukaran pesan berformat JSON, di mana kehilangan satu byte saja dapat merusak file atau menggagalkan proses decoding data.

WebSocket & HTTP : Browser memiliki batasan keamanan *sandbox* yang tidak mengizinkan koneksi langsung ke *raw TCP socket*. Oleh karena itu, digunakan protokol WebSocket (melalui library Socket.IO) untuk komunikasi dua arah (*full-duplex*) berlatensi rendah secara real-time (untuk chat dan menerima frame streaming video). Sementara itu, protokol HTTP digunakan secara spesifik untuk melayani unduhan file fisik dari direktori server via Flask route `/download/<filename>` secara asinkron.

SMTP (Simple Mail Transfer Protocol) : Merupakan standar industri yang andal dan aman untuk mengirimkan surat elektronik dari server ke penyedia kotak masuk email klien.

2. - Inisiasi & WebSocket: Browser Client (klien) terhubung ke Web Gateway (Flask) melalui protokol WebSocket.

- Koneksi TCP Internal: Setiap kali sesi browser baru terhubung, Web Gateway secara otomatis membuka koneksi raw TCP socket baru ke Server utama (server.py pada port 5050). Gateway memetakan ID sesi WebSocket (request.sid) ke socket TCP tersebut.

- Protokol Framing Custom: Agar data TCP tidak terpotong (*fragmentation*) di perjalanan, komunikasi antar socket dibungkus dengan header panjang data 4-byte.

3. - Socket TCP Putus: Blok try-except di sisi Server dan Gateway menangkap pemutusan mendadak (BrokenPipeError), segera menghentikan thread video streaming aktif klien tersebut, dan menutup koneksi dengan aman.

- Notifikasi Web: Gateway mengirimkan event `connection_lost` ke browser untuk menampilkan peringatan pop-up dan mengarahkan kembali ke halaman login.

4. Kelebihan : Transmisi Data yang Reliabel: Penggunaan framing length-prefixed dan transport TCP menjamin tidak ada pesan chat yang hilang dan file biner yang ditransfer selalu utuh.

Kekurangan :

- Single Point of Failure: Web Gateway (web_client.py) menjadi titik kegagalan tunggal. Jika proses gateway mati, seluruh browser klien tidak akan bisa terhubung ke server meskipun server TCP utama dalam keadaan aktif.
- *Overhead Bandwidth Video*: Pengiriman video frame-by-frame dengan format gambar JPEG base64 di atas WebSocket memakan bandwidth yang cukup besar.

- 5.

Keamanan: Menggunakan sertifikat SSL/TLS (HTTPS & WSS) untuk enkripsi pertukaran data end-to-end.

Kecepatan: Mengganti streaming JPEG base64 dengan protokol WebRTC (UDP) untuk latensi yang jauh lebih rendah dan efisiensi bandwidth yang tinggi.

Keandalan: Menambahkan mekanisme Auto-Reconnect dengan Exponential Backoff pada browser client jika terjadi pemutusan koneksi internet sementara.